

An experience report on detecting and repairing software architecture erosion

Francesca Arcelli Fontana*, Riccardo Roveda*, Marco Zanoni*, Claudia Raibulet*, Rafael Capilla†

*Department of Informatics, Systems and Communication

University of Milano-Bicocca, Milano, Italy

† Rey Juan Carlos University, Madrid, Spain

{arcelli, riccardo.roveda, zanoni, raibulet}@disco.unimib.it, rafael.capilla@urjc.es

Abstract—Architecture erosion constitutes the most visible effect of the degradation of design. It is a major reason to address the design debt, often caused by architectural mismatch problems. Today, the identification of design erosion is a major concern for designers and software maintainers. Adequate tools are necessary to identify and repair the debt. This paper describes our experience on identifying architectural erosion problems on existing open source software applications through the support of two well known tools. Moreover, we outline if the tools provide useful hints in repairing some of the detected problems.

I. INTRODUCTION

Systems erode over time when they evolve [1], and the challenge to keep the intended architecture aligned with code is still hard when software engineers must deal with obsolescence and maintenance. In order to combat obsolescence and rigidity of designs, the metaphor of Technical Debt (TD - a.k.a. design debt) introduced by Cunningham [2] to explain the causes and need of refactoring, covers an important role for software maintenance. Design debt concerns with the design evolution and trade-off decisions made during the architecting and implementation phases, and has to be repaid during maintenance. Today, the ways for identifying the most visible TD symptoms are several [3], [4]. Software designers must identify and estimate the size of the debt to be repaid to yield more sustainable architectures [5]. As stated by Letouzey and Ilkiewicz [6], there is no exact method to estimate TD because its calculation should be based on all the violations to be fixed. Nevertheless, the cost of fixing design violations sometimes exceeds the allocated budget. Thus, the estimated cost of the debt is often based on the severity of the design/code violations and the cost of their repair.

Most methods attempt to estimate the debt and software quality based on source code analysis, often related to specific quality characteristics [7], but there is a need for more approaches focused on identifying and estimating the debt at the architecture level. Different tools provide support in this direction, even if they often offer too much information. For example, a huge number of different quality metrics is often reported. Their interpretation is difficult, and deciding how to improve the system by simply relying on the metrics values is very hard. Moreover, many of the proposed views are often completely incomprehensible. It would be useful if the tools could provide some kind of prioritization of the problems

to be removed and suggestions to improve the architectural degradation. Among the different problems, we can identify, e.g., code smells, architectural smells and possible relations among them that could be investigated to find the patterns of smells leading to architectural problems [8], [9]. Code smells are symptoms of possible code or design problems that can be removed through the right refactoring steps [10], while architectural smells are usually introduced through the violation of some known design principles [11], [12].

In this paper, we describe our experience on the support provided by tools towards the identification of architectural erosion problems [13] and their solution. In particular, we experimented well known product families for software quality assessment, i.e., Sonargraph [14] and Structure101 Studio [15]. Other tools are available. We cite some of them and motivate our choice of these two tools in the next section.

With our experimentation, we aim to answer the following research questions:

- RQ1 How the tools differ in supporting the reconstruction of systems where a reference architecture is available or not (known/unknown system)?
- RQ2 Do the suggested restructuring actions effectively help in the simplification of the system?
- RQ3 Do the provided Technical Debt measures reflect the effort spent in restructuring the system?

We decided to apply the tools by analyzing a FLOSS (Free, Libre and Open Source System) system (Ant v1.8.4) and an application we have developed for the crawling and analysis of FLOSS projects, called RepoFinder [16]. By analyzing a system that we have developed, we are able to better capture the functionalities of the tools in reconstructing the software architecture when a reference architecture is available and known, in better identifying the problems and applying the right suggestions to remove them. Hence, we describe our experience in detecting problems that could cause architecture degradation, and in repairing some of these problems directly through the support of Sonargraph and Structure101, by removing for example cyclic dependencies, or by manually changing the code to remove some architectural violations.

The remainder of this paper is structured as follows. In Section II, we cite some of the tools we can use to identify architectural problems; in Section III, we describe our experimentation in detecting and repairing architecture erosion prob-

Table I
TOOLS FEATURE OVERVIEW

Tools	Metrics	Code Smells	Arch. Smells	Antipatterns	TDI	Refactoring Sugg.s	Warn. Violation	Cluster View	Arch. Violation
STAN [17]	44	0	2	1	No	No	Yes	Partial	No
MAE [18]	10	0	1	5	No	No	No	Partial	No
inFusion [19]	61	24	3	0	Yes	No	Yes	No	No
Lattix [20]	9	0	1	0	No	No	Yes	No	No
JDepend [21]	6	0	1	0	No	No	No	No	No
SA4J [22]	5	0	1	7	No	No	No	No	No
Understand [23]	47	1	0	0	No	No	Yes	No	No
Bauhaus [24]	12	2	0	0	No	No	No	No	No
IntelliJ IDEA [25]	250	9	0	0	No	Yes	No	No	No
CodePro Analytix [26]	35	2	0	0	No	Yes	Yes	No	No
SonarQube [27]	35	2	1	0	Yes	No	Yes	No	No
Sonargraph [14]	103	15	1	0	Yes	Yes	Yes	No	Yes
Structure101	6	0	1	0	Partial	Yes	Yes	Yes	Yes
Studio [15]									

lems with Sonargraph and in Section IV with Structure101. In Section V, we discuss the advantages or limits of the two tools. Finally, in Section VI we report our conclusions and future work.

II. RELATED WORK: TOOL SUPPORT FOR MANAGING ARCHITECTURE EROSION

Several tools have been developed, providing different functionalities that can be used to evaluate architecture degradation. They provide support for software analysis in different ways and collect different data. Some tools are proprietary and some are open source. In Table I, we list well known tools which offer different functionalities useful for software quality assessment and for identifying architectural debt. We included only tools that can be freely used, or where a free trial version is available. We excluded academic tools that are not available or not mature enough to be experimented without the authors' guidance.

In Table I, we have considered the following features:

- *Metrics*: the number of metrics supported by the tool.
- *Code Smells*: the number of detected code smells as defined, e.g., by Fowler [10] or Lanza and Marinescu [28].
- *Architectural Smells*: the number of detected architectural smells [11], [12]; examples of architectural smells are Cyclic Dependencies among modules or packages, Unstable Dependencies, Stable Abstraction Breaker (as in inFusion).
- *AntiPatterns*: the number of detected structural antipatterns [29], [30], e.g., Breakable, Butterfly, Hub; we do not consider the Tangle antipattern here, because the same concept is covered by the Cyclic Dependencies architectural smell.
- *Technical Debt Index*: if the tool provides some kind of Technical Debt (TD) or quality index giving an overall evaluation of the analyzed system.
- *Refactoring Suggestions*: if the tool supports refactoring and suggests the way to solve the identified problems.
- *Warning Violations*: if the tool provides some contextual warning of the detected violations.

- *Clustered View*: if the tool provides an automatic clustered view of the system architecture, grouping together tied components and displaying them in a graphical view.
- *Architectural Violations*: if the tool is able to detect some kinds of violations of a specified architecture.

We considered the above features because they can be useful for our analysis, but we did not obviously cited in the table all the tools computing metrics, or detecting code smells, or providing the same or similar features; many other tools exist, but we included tools providing more than one of the features outlined Table I.

An interesting characteristic of the tools we report in Table I is that no one of them uses historical information regarding the system to discover architectural issues. Many tools, instead, allow recording analysis snapshots and track them over time, to understand the trend in the quality of the analyzed project.

Other academic tools are being developed, and it will be interesting to understand their contributions when they will be available. Some examples are Hotspot Detector [31] and SCOOP [32]. A notable feature of Hotspot Detector is that it considers also the project historical information to detect architectural issues.

In a previous short paper [33], we experimented three of the reported tools (SonarQube, Sonargraph, inFusion), with the aim of understanding their capabilities in assessing the architecture of a system, with particular focus on the provided metrics for the estimation of Technical Debt. Sonargraph has been recognized as the tool (among the three) with the larger feature set for the task.

In this paper, we analyzed the architecture of two projects, and attempted their re-engineering according to the tools' suggestions. We decided to use Sonargraph, as we recognized it as the best tool in our previous work [33], extending the experimentation to its re-engineering features. The second chosen tool is Structure101 Studio, since it is specialized in architectural evaluation and re-engineering support, and takes a different approach than Sonargraph in both the analysis and re-engineering phases. Structure101 is also the only considered tool that provides an automatically reconstructed view of the

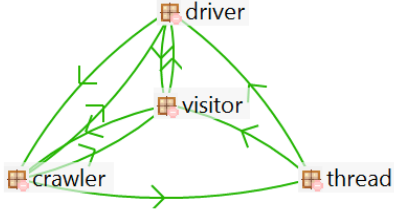


Figure 1. Sonargraph - Example of cycle to cut in the Cycles View on RepoFinder

architecture that clusters related components, with the aim of simplifying the comprehension of the system.

Few research works in the literature report experiences to control architecture erosion by evaluating the support provided by different tools, while we argue that they are useful to understand the directions for future research in this area. As an example, Gorton and Zhu [34] experiment five tools (Understand for Java, JDepend, SA4J, ARMIN, Enterprise Architect) on an industrial Java system composed of 50kLOC. After reconstructing the architecture with each tool, they identify different general and specific points for the improvement of the tools. In our work, we experiment two state-of-the-art and very complete tools, and we identify how they complement each other and the possible improvement directions.

III. DETECTING AND REPAIRING DESIGN EROSION WITH SONARGRAPH

We describe in this section our experience in detecting and repairing some architectural problems with Sonargraph on the two systems, RepoFinder and Ant. The size of the two systems can be found in Table II. We describe below some of the principal features and functionalities offered by Sonargraph respect to our aims. We also evaluate the evolution of these features (see Table II), as indicators of possible architecture erosion, before and after the refactoring steps.

Sonargraph checks the conformance of the software architecture with respect to a predefined model and finds possible violations in the project. The logical architecture can be defined using horizontal layers and vertical slices, each containing some code artifact. The suggested way of interpreting and defining layers and slices is: horizontal layers should be associated to technical aspects, e.g., the management of database or network access, while vertical slices to domain aspects, e.g., the concerns handled by the system. It is also important to define external libraries and subsystems, to better understand the system and how it relates to the external components. Packages and classes have to be assigned to layers, slices, subsystems, libraries and external components. This assignment is realized by specifying wildcard/glob patterns to match the names of packages and classes.

Sometimes, internal components are made for common usages like “database manager” (DBM), where the database must be accessed by a single component and the other components have to use it to access the database. In order to verify

Name Property	RepoFinder		Ant	
	before	after	before	after
Structural Debt Index	25,698	11,080	4,959	4,895
Architecture Violations	1,073	437	2	0
Cyclic Packages	31	22	30	25
Biggest Package Cycle Group	9	9	26	25
Code Smells	5,265	2,902	5,338	5,324
Code Smells/LOC	0.08	0.05	0.05	0.05
Packages	116	124	70	70
Classes	912	819	1,278	1,278
Lines of Code	67,536	58,300	105,007	105,054

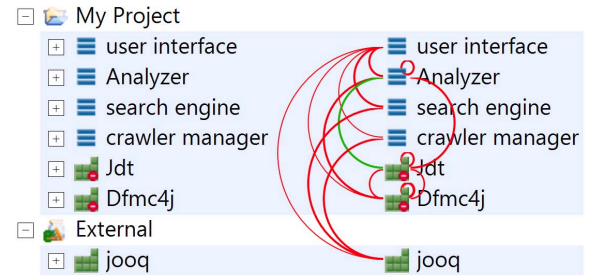


Figure 2. Sonargraph - Exploration view on RepoFinder

automatically if this practice is respected, we need to create “allowed dependencies” from the DBM to the database driver and from other components to the DBM. Otherwise we need to define “denied dependencies” from all components except for DBM to the database. After the definition of the allowed dependencies, defining denied dependencies is redundant.

Sonargraph provides a concise dependency graph called Exploration view, which shows the allowed, denied and cyclic dependencies, between components, external components (e.g., libraries) and packages. A Cyclic package group is shown in the Cycles view (see Figure 1), where packages are nodes and the edges represent dependencies.

The information extracted from the source code and the architecture is synthesized in a single measure, called Structural Debt Index. It has the aim of being roughly proportional to the effort needed to clean up structural quality problems in the code. The index consists of a score calculated assigning a different number of points to different elements:

- The number of type dependencies that would need to be cut to break all cyclic package dependencies: 10 points are scored for each type dependency and 1 additional point per occurrence of the dependency in the code (preferring dependencies that are also architecture violations).
- Architecture violations: 10 points are scored for each forbidden type dependency and 1 point per occurrence in the code (already cut cyclic architecture violating dependencies are not counted again).
- Refactoring tasks: 1 point scored for each move, delete and rename refactoring task, 10 points per ‘cut depen-

dency' refactoring task. For delete and cut refactoring tasks, 1 additional point per occurrence in the code.

The Structural Debt Index is calculated as the sum of the obtained points. Another measure, called Structural Debt Cost, is computed by assigning a time to each point of the Structural Debt Index. The suggested time, as an initial setting, is between 6 and 10 minutes for each point. It is possible to define programmers' hour costs and to obtain an estimation of the cost of correcting the issues in the system.

Other features of the tool are the detection of duplicated code and the extraction of metrics. Duplicated code is another indicator of refactoring opportunities, and is summarized in the Duplicated code view. The 103 metrics computed by the tool are categorized and visualized using a tree view displaying all the granularity levels, e.g., layers, packages.

A. Detecting Design Erosion

RepoFinder is a project that we designed and developed. For this reason, we were able to fully specify its reference architecture in Sonargraph. Table III reports the rules used to group packages in components in RepoFinder. “*” means that every class and interface directly contained in the package is considered. “**” considers every subpackage and contained classes or interfaces.

We found several architectural violations in RepoFinder (1073), as shown in Table II. We checked them and we observed that the most frequent ones are related to the database usage. The only component allowed to access the database library (JOOQ) is the database manager, but we found accesses from other components, e.g., directly reading tables through JOOQ. The second most frequent architecture violation is related to the direct access to the Apache Lucene index. This, in fact, has to be accessed only by the crawling and indexing components of RepoFinder.

We also found a large number of smells. For code smells, Sonargraph identifies few code smells defined by Fowler and other code anomalies of different kinds and granularity, as unused/missed/hideable-something. In particular, we found 4987 code smells, with a high number of unused methods (2985), hideable public methods (600), missed abstraction methods (522), unused classes unreachable (452).

Moreover, in Table II we outline a number of Cyclic Packages detected in RepoFinder and the biggest package cycle, which is composed by 9 packages. Cyclic packages, also called Cyclic Dependency, represent one of the most critical architectural smell [11], [12].

Sonargraph requires the definition of the reference architecture of the system, to provide the detection of architectural violations. For Ant, architectural documentation is very scarce. We found a reference to the separation of packages in modules in the Javadoc contained in the project's binaries¹. In the documentation, allowed and disallowed dependencies are not defined. The only information available is the division of packages in modules.

¹<http://archive.apache.org/dist/ant/binaries/apache-ant-1.8.4-bin.zip>

Table III
REPOFINDER COMPONENTS DEFINITION PATTERNS

Package	Component
it.unimib.disco.essere.serial.**	search engine
it.unimib.disco.essere.serial.*	search engine
it.unimib.disco.essere.serial	search engine
it.unimib.disco.essere.analysis.**	analyzer
it.unimib.disco.essere.analysis.*	analyzer
it.unimib.disco.essere.analysis	analyzer
it.unimib.disco.essere.repofinderUI.tool.**	analyzer
it.unimib.disco.essere.repofinderUI.tool.*	analyzer
it.unimib.disco.essere.repofinderUI.tool	analyzer
it.unimib.disco.essere.dashboard.**	user interface
it.unimib.disco.essere.dashboard.*	user interface
it.unimib.disco.essere.dashboard	user interface
it.unimib.disco.essere.repofinderUI.storage.**	db manager
it.unimib.disco.essere.repofinderUI.storage.*	db manager
it.unimib.disco.essere.repofinderUI.storage	db manager
org.apache.lucene.**	apache
org.apache.lucene.*	apache
org.apache.lucene	apache
org.jooq.**	jooq
org.jooq.*	jooq
org.jooq	jooq
org.jsoup.**	jsoup
org.jsoup.*	jsoup
org.jsoup	jsoup
net.sourceforge.pmd.**	pmd
net.sourceforge.pmd.*	pmd
net.sourceforge.pmd	pmd
jdepend.xmlui.**	jdepend
jdepend.xmlui.*	jdepend
jdepend.xmlui	jdepend
com.puppycrawl.tools.checkstyle.**	checkstyle
com.puppycrawl.tools.checkstyle.*	checkstyle
com.puppycrawl.tools.checkstyle	checkstyle
com.aragost.javahg.**	hg
com.aragost.javahg.*	hg
com.aragost.javahg	hg
com.vaadin.**	vaadin
com.vaadin.*	vaadin
com.vaadin	vaadin
org.tmatesoft.svn.core.**	svn
org.tmatesoft.svn.core.*	svn
org.tmatesoft.svn.core	svn
org.eclipse.jgit.**	git
org.eclipse.jgit.*	git
org.eclipse.jgit	git
javacss.**	javacss
javacss.*	javacss
javacss	javacss
jxl.**	jxl
jxl.*	jxl
jxl	jxl

The packages of Apache Ant 1.8.4 have been grouped in five components: core, task, types, util and tools. Table IV summarizes the rules used to group packages.

Sonargraph reported only two architectural violations in Ant (see Table II), where two classes located in sub-packages of `org.apache.tools.ant` access classes outside the ant package, e.g., classes contained in `org.apache.tools.zip`. Compared to RepoFinder, Ant has lower Structural Debt Index but contains a much larger package cycle group (26 packages). The number of identified code smells in the two systems is comparable (5338 vs 5265), while code smell density is higher in RepoFinder.

Table IV
ANT COMPONENTS DEFINITION PATTERNS

Package	Component
org.apache.tools.ant	Core
org.apache.tools.ant.*	Core
org.apache.tools.ant.taskdefs	Task
org.apache.tools.ant.taskdefs.*	Task
org.apache.tools.ant.taskdefs.**	Task
org.apache.tools.ant.taskdefs.optional	Task
org.apache.tools.ant.taskdefs.optional.*	Task
org.apache.tools.ant.taskdefs.optional.**	Task
org.apache.tools.ant.types	Types
org.apache.tools.ant.types.*	Types
org.apache.tools.ant.types.**	Types
org.apache.tools.ant.types.optional	Types
org.apache.tools.ant.types.optional.*	Types
org.apache.tools.ant.types.optional.**	Types
org.apache.tools.ant.util	Util
org.apache.tools.ant.util.*	Util
org.apache.tools.ant.util.**	Util
org.apache.tools.bzip2	Tools
org.apache.tools.bzip2.*	Tools
org.apache.tools.bzip2.**	Tools
org.apache.tools.mail	Tools
org.apache.tools.mail.*	Tools
org.apache.tools.mail.**	Tools
org.apache.tools.tar	Tools
org.apache.tools.tar.*	Tools
org.apache.tools.tar.**	Tools
org.apache.tools.zip	Tools
org.apache.tools.zip.*	Tools
org.apache.tools.zip.**	Tools

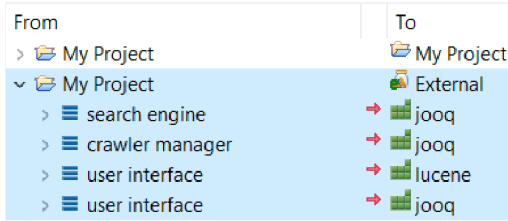


Figure 3. Sonargraph - Architecture Violations in RepoFinder

B. Repairing Design Erosion

In order to highlight architectural violations, Sonargraph provides the “Exploration View”, which shows dependencies using colored edges: green for correct dependencies and red for violated dependencies. Figure 2 shows the view over RepoFinder.

In RepoFinder, we followed the indications about violated dependencies reported on the use of database libraries and we removed them. In the logical architecture of the project, we defined the JOOQ library as an external sub-system. The Architecture Violations view lists the existing violations (see Figure 3). We started from the information of this view to perform our refactoring. The refactoring operations can be summarized with three refactoring actions: (i) the first is to create an external component dedicated to the use and management of the database, (ii) the second one is to eliminate, where it is possible, all usages and references to the JOOQ subsystem from all the components, and (iii) the third one is

to use the new database manager component to access and use the JOOQ library. These actions significantly lowered the Structural Debt Index of the project, by more than a half.

In Ant, we solved the two reported architectural violations. In order to solve these violations, we moved the methods from the called class to the calling class, removing the dependency between the packages.

C. Maintainability and Evolution using SonarGraph

In Table II, we reported the evolution of the data collected before and after repairing some of the identified problems in the two systems.

In RepoFinder, we can observe that the number of Architecture Violations and Cyclic Packages, and the Structural Debt Index decreased significantly. The three values are tied, indeed, since the Index is computed over the other two measures. While code smells are not directly related to the applied refactoring, their number changed drastically: unused methods lowered by a half ($5265 \rightarrow 2860$), hideable public methods by a half ($600 \rightarrow 301$), missed abstraction methods by a tenth ($522 \rightarrow 66$); unused classes unreachable did not lower so drastically ($452 \rightarrow 398$). Lines of Code significantly decreased, too.

In Ant, our refactoring actions slightly decreased the Structural Debt Index. We tried also to solve cyclic dependencies, exploiting the recommended dependency cuts proposed by Sonargraph. However, after solving five cycles, we realized that this operation was not effective in tackling the measured Structural Debt. The number of code smells decreased by only 14 code smells, of these types: unused attributes, unused methods.

Considering the refactoring actions executed on the two systems, we can assert that the Structural Debt Index has been proportional to the effort spent. The reason why we did not apply more changes to Ant is mainly that the tool did not provide many feasible refactoring opportunities. Some relevant proposed changes were extremely complex, and we did not address them. The tool is certainly useful both for detecting and repairing architecture defects and monitor the quality of an application during maintenance. However, on a project where the architecture is not known in much detail, the proposed refactoring opportunities can be not effective or very difficult to address.

IV. DETECTING AND REPAIRING DESIGN EROSION WITH STRUCTURE101

In this section, we execute the same analysis performed with Sonargraph by exploiting Structure101 Studio for Java v4.2.10071 (Structure101 in the following) [15]. The tool provides different views, with the ability to define, inspect and manipulate the model of the analyzed system, to show the collaboration between code items, understand the class hierarchy, navigate the call method graph and detect Fat items and/or Tangles.

Fat corresponds to complexity measured on a single item, at any level of granularity. Cyclomatic Complexity (CC)

is used to measure Fat on methods, while at any other level of granularity Fat is measured as the number of inter-dependencies existing among the item's contents. For example, Fat in packages is the number of dependencies among the contained classes. Size measures are not used to measure Fat, since it is assumed that the complexity of the control flow or of the structure makes an item hard to understand, and not, e.g., its number of sequential lines of code. Items are considered "Fat" when their Fat value exceeds a defined threshold. For example, the default threshold for methods is 15.

Tangles are cyclic, package-level dependencies. As widely known, they drive up the coupling of a structure, making it much harder to understand, extend or modify. Moreover, activities such as regression testing, software reuse, and porting to modern frameworks become virtually impossible. Structure101 organizes Tangles into levels by calculating the Minimum Feedback Set (mfs), i.e., a minimum set of dependencies that, if removed, would render the dependency graph acyclic, and by showing the levels as if the mfs dependencies were removed. Tangles are computed at the package and design granularity levels. Each package or component is assigned to a Tangle percentage, reflecting the portion of its content participating to a tangle.

By using Fat, Tangle, and a Size metric (LOC), Structure101 computes another metric called XS ("excess"), on every item: methods, classes, packages, components. XS represents an estimation of the portion of the LOC of an item that are "affected" by Fat or Tangle. For example, if a package has 33% Tangle and 1000 LOC, its XS value will be 330; if a method has Fat = 20 (with threshold 15) and 100 LOC, its XS will be $(20 - 15)/20 \cdot 100 = 25$. XS is provided to allow the user to drill down on a code-base to discover areas of over-complexity, helping to reach a prioritization of the refactoring efforts. All items exceeding their respective Fat or Tangle thresholds are considered as *XS offenders* in Structure101. Because of the definition of the metric, only XS offenders have XS values larger than 0.

A. Detecting Design Erosion

We used Structure101 to analyze the software architecture of Ant 1.8.4 and RepoFinder. Structure101 reconstructs the software architecture and shows the top list of *Tangles* and *Fat* elements. The structural over-complexity graph, shown in Figure 4², shows the percentage of the elements of the system are considered "Tangled" or "Fat". The graph classifies a system as structured or unstructured. If the percentage of Tangled and Fat items in the system is near to 0%, the system is structured, while if it is near to 100% the system is considered unstructured. The graph shows how much structure the system has, and how much it will have after the planned refactoring actions.

The reconstructed architecture is shown using the Lev-elized Structure Map (LSM), which allows choosing different grouping methods: cohesive cluster, tangles and tagged items.

²the background color is not relevant to understand the figure

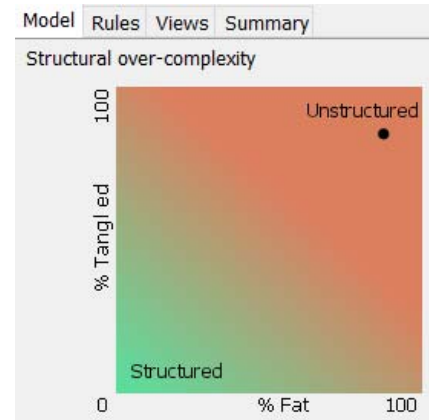


Figure 4. Structure101 - Structural over-complexity view on Ant

Figure 5 shows Ant's LSM, applying the Cohesive Cluster grouping method. System items are grouped according to their common dependencies and LSM levels. It helps to understand Fat regions, and can help in the process of subdividing a Fat item in several smaller items. The large red/dark rectangular group depicted in Figure 5 is a tangle of 48 items, contained in a clustered group of 56 items belonging to the `org.apache.tools.ant` package. There is also an orphan item (`MagicNames` class) that is not referred by items of its package. Items can be tagged, simplifying their comprehension process, by allowing to easily locate them in different views and to understand their collaboration with the rest of the system.

Defining the logical architecture of the system is important to check possible architecture violations. This feature is provided by Structure101 in the rule tab shown in Figure 6. It is possible to define more than one diagram and check different configurations of the system architecture, counting how many class dependencies violations there are w.r.t. the designed one. The diagram defines layering through the top-down arrangement of cells. In general, cells may only be used by others in higher layers, but it is possible to define exceptions and to override this rule, by drawing green (continuous line) arrows (to allow dependencies) and red (dashed line) arrows (to disallow dependencies). Black arrows are reported only when there is a violation among the designed components.

Other views exist for the detailed evaluation of Tangles or Fat items, to focus attention on tangles and their elements, to show the collaboration among different source items, and display an overview of the hierarchical composition of code and call graphs.

Structure101 found 3 tangles at package level, and 26 at class level in Ant (see Table V). It also found 29 class-to-class dependency violations from the defined architecture.

RepoFinder (see Table V) contains more tangles than Ant, but they are smaller: the largest package tangle in RepoFinder contains 9 classes, while in Ant the largest tangle contains 26 classes. The result is consistent with the one reported by Sonargraph and described in Section III.

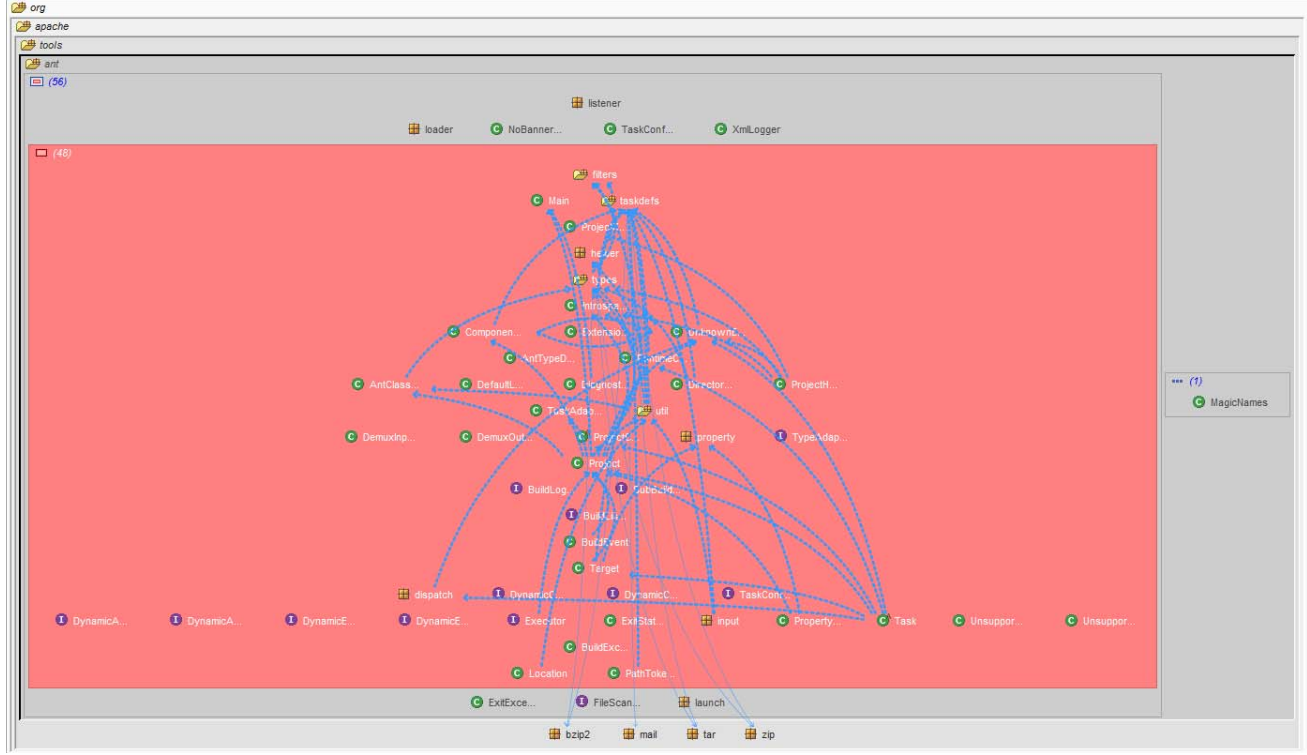


Figure 5. Structure101 - Exploration View on Ant

Table V
STRUCTURE101 - DETECTED TANGLES

Level		Ant		RepoFinder	
		before	after	before	after
Package	#Items	70	70	308	289
Package	#Tangles	3	2	9	7
Package	#Tangled items	30	28	30	25
Package	Biggest	26	26	9	9
Class	#Items	845	850	1,467	1,288
Class	#Tangles	26	14	39	13
Class	#Tangled items	300	272	296	200
Class	Biggest	181	181	74	74

Table VI
DENSITY OF TANGLED AND FAT ITEM IN ANT (OF. ABBREVIATION OFFENDER)

		Threshold	before	after
Tangled (design)	#Of.s	0	9 of 17	8 of 17
Tangled (design)	%Of.s	0	53%	47%
Tangled (design)	%XS	0	63%	63%
Fat (design)	#Of.s	120	0 of 17	0 of 17
Fat (design)	%Of.s	120	0%	0%
Fat (design)	%XS	120	0%	0%
Fat (package)	#Of.s	120	2 of 70	2 of 70
Fat (package)	%Of.s	120	3%	3%
Fat (package)	%XS	120	11%	12%
Fat (class)	#Of.s	120	16 of 1,270	16 of 1,270
Fat (class)	%Of.s	120	1%	1%
Fat (class)	%XS	120	13%	14%
Fat (method)	#Of.s	15	95 of 11,205	95 of 11,205
Fat (method)	%Of.s	15	1%	1%
Fat (method)	%XS	15	12%	12%

Table VII
DENSITY OF TANGLED AND FAT ITEM IN REPOFINDER (OF. ABBREVIATION OF OFFENDER)

		Threshold	before	after
Tangled (design)	#Of.s	0	16 of 202	14 of 190
Tangled (design)	%Of.s	0	8%	7%
Tangled (design)	%XS	0	69%	66%
Fat (design)	#Of.s	120	0 of 202	0 of 190
Fat (design)	%Of.s	120	0%	0%
Fat (design)	%XS	120	0%	0%
Fat (package)	#Of.s	120	0 of 308	0 of 289
Fat (package)	%Of.s	120	0%	0%
Fat (package)	%XS	120	0%	0%
Fat (class)	#Of.s	120	17 of 1,816	16 of 1,614
Fat (class)	%Of.s	120	1%	1%
Fat (class)	%XS	120	17%	19%
Fat (method)	#Of.s	15	41 of 12,212	38 of 9,500
Fat (method)	%Of.s	15	0%	0%
Fat (method)	%XS	15	14%	15%

Tables VI-VII show the density of Tangle and Fat items in the system, and their respective thresholds, at all granularity levels. In Ant, 9 of 17 defined component are tangled. Tangles are the prominent XS violation for Ant. In fact, Tangles cover 63% all the XS measured on the system. The density of Fat items is generally low (1%–3%).

In RepoFinder, the density of Tangles is lower, but they still represent the largest part of the XS in the system. Fat items are present only at class level and method level, with density value of 0%–1%.

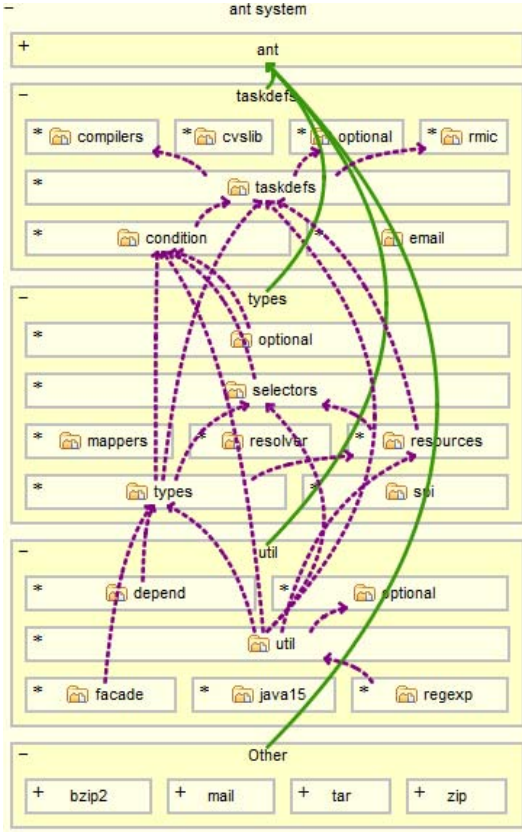


Figure 6. Structure101 - Architecture Diagram of Ant

B. Repairing Design Erosion

Structure101 builds a model of the analyzed system, containing, e.g., its structure, architectural violations, XS metric values and offenders. Each model is stored in a repository that can be accessed by different developers. A single repository can contain the models of several versions of several different systems.

To support the removal of tangles from source code, Structure101 provides an action plan, which is also stored in the model, containing the recommended refactoring to perform on the system, to apply the refactoring actions simulated through the tool. Every single action is an atomic source code change that involves one or two items (e.g., add class A or move method $m()$ from A to B).

The action plan can be accessed from the Eclipse IDE, but cannot be automatically applied. Developers must choose how and if to follow the action plan. The selection of an item in the action plan redirects to the file interested by the action. Structure101 does not provide a way to apply a refactoring automatically. Since Eclipse provides some refactoring automation, e.g., extract class with parameters, move method, it is possible to exploit them, but they are not triggered by the Structure101 plugin. Moreover, the action plan is not synchronized with the IDE. To update the action plan, a full project analysis must be executed.

In Ant, we resolved 12 tangles at class level and 1 at package level, removing dependencies in the Minimum Feedback Set, to reduce the refactoring effort. The resolution of tangles did not effectively enhance the structure of the system measured by the tool, probably due to the fact that only relatively small tangles were resolvable, and their resolution covered a small part of the system. While working on Ant, which is a project we do not know, especially about its design, we noticed that Structure101 is particularly suitable to understand the system without prior architecture knowledge. The tool provides a clear view of the system, and allows experimenting with different refactoring simulations through its interactive user interface.

In RepoFinder, we noticed the problem of database access, that was spread across the system components. We applied refactoring to move to the database component a lot of dependencies regarding the JOOQ library, similarly to what we did with Sonargraph. The tool provided a precise list of the refactoring actions, but the overall effort has been the same we spent with Sonargraph, because there is no automation of refactoring actions, and the additional information regarding clustering sets is confusing.

C. Maintainability and Evolution using Structure101

The measures reported by Structure101 on Ant (see Table VI) did not change significantly after the application of the refactoring actions. Most measures did not change their value at all, with the exception of the percentage of Tangle at design granularity. The number of Tangles has been decreased by refactoring actions performed at both class level and package level. Despite the resolution of 12 class Tangles and 2 package Tangles, the improvement in the structural measures has been very low.

The refactoring actions applied to RepoFinder lowered significantly its size, as shown in Table VII: number of methods (12.212 $\rightarrow$ 9.500), classes (1816 $\rightarrow$ 1614) and package (308 $\rightarrow$ 289). This reduction of the system was not proportionally followed by a reduction of Fat items, leading to a slight increase in the Fat percentage. This aspect is something probably worth noting, since Size does not influence Fat metrics, but a 22% change in the size of the system probably has an impact on its understandability. However, Size is always shown in the tool together with the other metrics, allowing the user to take both indicators into consideration.

V. DISCUSSION AND LESSONS LEARNED

According to our experimentation of Sonargraph and Structure101, we can observe that:

- The automatic clustering features provided by Structure101 are particularly useful to understand and analyze unknown systems. The detection of Tangles (or Cyclic Dependencies) is another feature relevant for this task, and is supported by both tools.
- The architecture definition functionalities of Sonargraph allow a more precise architecture specification. The effect of the specification, i.e., the assignment of items to

components, and the detected violations, can be inspected interactively.

- Action plans generated by Sonargraph contain high-level restructuring tasks, e.g., cutting the dependency between two classes. Structure101 gives the developer more support, providing a detailed action list that realizes the high-level action plan.
- The tools do not associate refactoring actions to time/-cost/effort measures. Moreover, the structural measures provided by the tool are not explicitly related to time or cost, and refactoring actions are not associated to the expected gain in the respective structural measures. This results in a lack of prioritization of the generated action plans.
- The action list provided by Structure101 is more detailed than the action plan generated by Sonargraph, and can be inspected in the IDE. Despite these advantages, there is space for improvement. A first issue to solve is the lack of interaction with the IDE: when a refactoring step coming from the action list is applied, it is not removed from the action list. On large action lists, this can lead to confusion and lack of manageability. A second issue is the fact that the action list items are not contextualized with the project. The selection of an item can lead to the relevant file, but not to the precise point of intervention, slowing down the application of refactoring.

Conducting this experimentation on detecting and repairing software architecture problems has given us considerable insights into various facets of the tools we experimented and how they tackle the essential problems that must be dealt with during an evaluation and reconstruction effort. We outline below the insights that in our opinion need still to be addressed:

- 1) as we observed, the tools we experimented, and those cited in Table I, do not support software architecture history analysis. This could be a relevant direction of improvement of the existing tools. Some techniques exist that exploit this historical information, e.g., the detection of evolutionary coupling can be used to discover unseen dependencies.
- 2) The tools we experimented do not allow to execute the planned refactoring actions automatically or semi-automatically. This would be of great value for developers, and a small gap to bridge for tool vendors. In fact, most IDEs support refactoring actions, and a contextualized action plan could easily trigger them in the IDE, leaving to the developer only the effort of deciding details like the name of the new classes or methods to be created.
- 3) Tools do not address the dynamic analysis of the system yet. This aspect of dynamic analysis is surely more difficult to tackle, but has the potential of revealing very important information about the connections among the components of the system, especially in distributed or service-oriented architectures.

- 4) The only architectural smell detected by the two tools is Cyclic Dependencies (Tangle). Many other architectural smells have been defined [12], [31], addressing very different properties of the software architecture.

Among these insights and those outlined by Gorton and Zhu [34], we can observe that fortunately many of the issues that they outlined more than 10 years ago have been solved or partially addressed. One issue that remains open is related to the above point 3, i.e., the exploitation of dynamic information is still not popular in mainstream tools despite the large amount of research in the dynamic analysis field [35].

VI. CONCLUSION AND FUTURE WORK

In this paper, we have described our experience using Sonargraph and Structure101 to detect code and architecture problems and in repairing some of them. We answer below to the research questions that we introduced in Section I.

RQ1 - How the tools differ in supporting the reconstruction of systems where a reference architecture is available or not (known/unknown system)?

Both tools allow specifying the reference architecture of the analyzed project. We found Sonargraph more mature and richer in terms of offered functionalities. Its way of specifying the reference architecture of the system is more detailed, allowing, e.g., to specify layers and slices, and to interactively inspect the packages and classes contained in the specified components. Structure101 handles the component definition mainly relying on the existing packages of the system, which can only be grouped graphically. Wildcard patterns can be used to exclude unwanted items from the architecture by name, but not to define components as in Sonargraph. Components can be organized in a hierarchy, but there are no concepts similar to Sonargraph's layers and slices.

As for the support for architecture evaluation and discovery, without the specification of the reference architecture, the two tools have different characteristics. Structure101 allowed us to discover more relevant architectural issues on Ant, even without the definition of its architecture. In particular, the Cohesive Cluster grouping method used in the LSM view is particularly effective in highlighting different architectural aggregates, without any prior knowledge. For example, it allowed to discover an architectural violation regarding database access, without the need of defining the reference architecture of the system. Sonargraph, instead, provides the detection of cyclic dependencies, but for any other evaluation it fully depends on the definition of the reference architecture of the system. For these reasons, we argue that Structure101 provides better support in the case of unknown architecture.

RQ2 - Do the suggested restructuring actions effectively help in the simplification of the system? In both the tools, the reconstructing actions chosen effectively helped in the simplification of the system, in proportion to the needed effort. Both tools do not provide suggestions regarding the priority of the actions to take. Sonargraph allows the engineer to assign a level of priority to each selected refactoring action. The action plan produced by Structure101 is ordered according to the

sequence of operations selected in the tool. Sonargraph sorts the action plan items first by priority and then according to the sequence of operations selected in the tool. In both tools, there is no automatic suggestion of the most effective actions to undertake. Moreover, there is no way to trigger automatic refactoring of the code starting from an action plan.

RQ3 - Do the provided Technical Debt measures reflect the effort spent in restructuring the system? In both the tools, the provided measures, i.e., the Structural Debt Index and Tangle%/Fat%, in Sonargraph and Structure101 respectively, reflected the effort spent during the refactoring actions on the two systems. The two measures have different scales, but they are proportionally tied to the structural problems of the analyzed systems. However, in both tools it is not possible to reliably estimate the effort needed to execute the action plans.

We plan in the next future to experiment other tools, e.g., CAST and other tools cited in Table I by combining the features they provide. We would like also to analyze the usefulness of the tools to evaluate different quality attributes (e.g., maintainability, reliability and security) and define adequate strategies to prioritize the quality attributes that are more critical to address concrete architecture violations.

REFERENCES

- [1] J. V. Gorp, J. Bosch, and S. Brinkkemper, "Design erosion in evolving software products," in *Proceedings of the International Workshop on Evolution of Large-scale Industrial Software Applications (ELISA)*. Amsterdam, The Netherlands: ESF RELEASE research network, Sep. 2003, pp. 134–139, co-located with ICSM 2003.
- [2] W. Cunningham, "The wycash portfolio management system," *OOPS Messenger*, vol. 4, no. 2, pp. 29–30, 1993.
- [3] R. Nord, I. Ozkaya, P. Kruchten, and M. Gonzalez-Rojas, "In search of a metric for managing architectural technical debt," in *Proceedings of the 2012 Joint Working IEEE/IFIP Conference on Software Architecture (WICSA) and European Conference on Software Architecture (ECSA)*. Helsinki, Finland: IEEE, Aug. 2012, pp. 91–100.
- [4] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
- [5] H. Koziol, D. Domis, T. Goldschmidt, and P. Vorst, "Measuring architecture sustainability," *IEEE Software*, vol. 30, no. 6, 2013.
- [6] J. Letouzey and M. Ilkiewicz, "Managing technical debt with the SQALE method," *IEEE Software*, vol. 29, no. 6, pp. 44–51, Nov 2012.
- [7] ISO/IEC, "ISO/IEC 25010 - Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models," Tech. Rep., 2010.
- [8] F. A. Fontana, V. Ferme, and M. Zanoni, "Towards assessing software architecture quality by exploiting code smell relations," in *Proceedings of the 2nd IEEE/ACM International Workshop on Software Architecture and Metrics (SAM 2015)*, Florence, Italy, May 2015, pp. 1–7.
- [9] I. Macia Bertran, R. Arcoverde, A. Garcia, C. Chavez, and A. von Staa, "On the relevance of code anomalies for identifying architecture degradation symptoms," in *Proceedings of the 16th European Conference on Software Maintenance and Reengineering (CSMR 2012)*, Szeged, Hungary, Mar. 2012.
- [10] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, USA: Addison-Wesley, 1999.
- [11] J. Garcia, D. Popescu, G. Edwards, and N. Medvidovic, "Toward a catalogue of architectural bad smells," in *Proceedings of the 5th International Conference on the Quality of Software Architectures (QoSA 2009)*. East Stroudsburg, PA, USA: Springer Berlin Heidelberg, Jun. 2009, pp. 146–162.
- [12] M. Lippert and S. Roock, *Refactoring in Large Software Projects: Performing Complex Restructurings Successfully*. Wiley, Apr. 2006.
- [13] L. de Silva and D. Balasubramaniam, "Controlling software architecture erosion: A survey," *Journal of Systems and Software*, vol. 85, no. 1, 2012.
- [14] hello2morrow, "Sonargraph Product Family," 2015, <https://www.hello2morrow.com/products/sonargraph>.
- [15] Headway Software Technologies, "Structure101," 2016, <http://structure101.com/products/>.
- [16] F. Arcelli Fontana, R. Roveda, and M. Zanoni, "Discover knowledge on FLOSS projects through RepoFinder," in *Proceedings of the International Conference on Knowledge Discovery and Information Retrieval (KDIR 2014)*. Rome, Italy: INSTICC Press, October 2014, pp. 485–491.
- [17] Odysseus Software GmbH, "STAN," 2015, <http://stan4j.com/>.
- [18] J. Dietrich, "Upload your program, share your model," in *Proceedings of the 3rd Conference on Systems, Programming and Applications: Software for Humanity (SPLASH '12)*. Tucson, Arizona, USA: ACM, Oct. 2012, pp. 21–22.
- [19] Intooitus, "inFusion," 2015, <http://www.intooitus.com/products/infusion>.
- [20] Lattix, Inc., "Lattix Architect and Lattix Analyst," 2015, <http://lattix.com/lattix-architect-and-lattix-analyst>.
- [21] Clarkware, "JDepend," 2015, <http://clarkware.com/software/JDepend.html>.
- [22] IBM Alphaworks, "SA4J — Structural Analysis for Java," 2004.
- [23] Scientific Toolworks, Inc., "Understand Your Code," 2015, <https://scitools.com/>.
- [24] A. Raza, G. Vogel, and E. Plödereder, "Bauhaus – a tool suite for program analysis and reverse engineering," in *Proceedings of the 11th Ada-Europe International Conference on Reliable Software Technologies (Ada-Europe 2006)*. Porto, Portugal: Springer Berlin Heidelberg, Jun. 2006, pp. 71–82.
- [25] JetBrains s.r.o., "IntelliJ IDEA," 2015, <https://www.jetbrains.com/idea/>.
- [26] Google, Inc., "CodePro Analytix User Guide," 2015, <https://developers.google.com/java-dev-tools/codepro/doc/>.
- [27] SonarSource S.A., "SonarQube," 2015, <http://www.sonarqube.org/>.
- [28] M. Lanza and R. Marinescu, *Object-Oriented Metrics in Practice*. Springer, 2006.
- [29] F. Arcelli Fontana and S. Maggioni, "Metrics and antipatterns for software quality evaluation," in *Proceedings of the 34th IEEE Software Engineering Workshop (SEW 2011)*. Limerick, Ireland: IEEE, Jun. 2011, pp. 48–56.
- [30] D. Binkley, N. Gold, M. Harman, Z. Li, K. Mahdavi, and J. Wegener, "Dependence anti patterns," in *Proceedings of the 23rd IEEE/ACM International Conference on Automated Software Engineering - Workshops*. L'Aquila, Italy: IEEE, Sep. 2008, pp. 25–34.
- [31] R. Mo, Y. Cai, R. Kazman, and L. Xiao, "Hotspot patterns: The formal definition and automatic detection of architecture smells," in *Proceedings of the 12th Working IEEE/IFIP Conference on Software Architecture (WICSA 2015)*. Montreal, QC, Canada: IEEE, May 2015, pp. 51–60.
- [32] I. Macia, R. Arcoverde, E. Cirilo, A. Garcia, and A. von Staa, "Supporting the identification of architecturally-relevant code anomalies," in *Proceedings of the 28th IEEE International Conference on Software Maintenance (ICSM 2012)*. Trento, Italy: IEEE, Sep. 2012, pp. 662–665.
- [33] F. Arcelli Fontana, R. Roveda, and M. Zanoni, "Tool support for evaluating architectural debt of an existing system: An experience report," in *Proceedings of the 31st ACM/SIGAPP Symposium on Applied Computing (SAC 2016)*. Pisa, Italy: ACM, Apr. 2016, to appear.
- [34] I. Gorton and L. Zhu, "Tool support for just-in-time architecture reconstruction and evaluation: an experience report," in *Proceedings of the 27th International Conference on Software Engineering (ICSE 2005)*. St. Louis, USA: IEEE, May 2005, pp. 514–523.
- [35] B. Cornelissen, A. Zaidman, A. van Deursen, L. Moonen, and R. Koschke, "A systematic survey of program comprehension through dynamic analysis," *IEEE Transactions on Software Engineering*, vol. 35, no. 5, pp. 684–702, Sep. 2009.